

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

<College / University Name>

PROJECT TITLE

UniVerify V2.0

A Decentralized Academic Certificate Verification Protocol Using Alphanumeric Keyspace Mapping

A Final-Year B.Tech Major Project — Submitted in partial fulfilment for the degree of B.Tech (CSE)

SUBMITTED BY

<Student Name>

<Roll Number>

<Branch / Semester>

PROJECT GUIDE

<Guide Name>

<Designation>

Dept. of CSE

Abstract

A one-paragraph summary of the problem, approach and result.

Conventional academic record verification depends on centralized databases and physical documents, introducing a single point of failure (SPOF), forgery risk, and verification latency of 7–14 days. UniVerify V2.0 is a decentralized verification protocol that anchors certificate records on the Ethereum Sepolia testnet through a Solidity smart contract.

The design replaces array/loop lookups ($O(N)$) with an explicit string-keyed state mapping on the Registration Number, achieving $O(1)$ constant-time retrieval. Role-Based Access Control freezes the deploying wallet as the sole issuer, and public verification is a gasless read-only view call via Ethers.js + MetaMask.

Phase 1 (current) validates the contract under a manual curation paradigm — data is entered by hand in Remix with a mock hash placeholder. Phase 2 will add an autonomous AI agent performing SHA-256 / IPFS hashing and programmatic transaction signing.

Why blockchain-anchored credentials, and what this project delivers.

■ **Problem domain.**

Degree and certificate verification is slow, manual and trust-dependent, relying on institutions to confirm authenticity.

■ **Blockchain premise.**

A public, append-only ledger provides tamper-evident, independently verifiable records without a central authority.

■ **Core idea.**

Store each certificate as a struct in a Solidity mapping keyed by the alphanumeric Registration Number for instant lookups.

■ **Access model.**

Only the university wallet can issue records; anyone can verify them for free through a read-only call.

■ **Scope of this report.**

Phase-1 prototype on Sepolia with manual data entry; Phase-2 automation roadmap is described as future work.

Literature Survey

Representative prior work and the gap UniVerify V2.0 addresses.

Reference / Year	Approach	Limitation / Gap
Nakamoto (2008)	Peer-to-peer electronic cash; proof-of-work ledger	General ledger; not tailored to credentialing
Buterin (2014)	Ethereum + programmable smart contracts	Enables apps but no credential schema provided
MIT Blockcerts (2016)	Open standard for blockchain certificates	Hash/issuer focus; lookup efficiency not emphasised
Grech & Camilleri (2017)	Survey of blockchain in education	Policy-level; lacks a concrete O(1) data model
IPFS – Benet (2014)	Content-addressed distributed file store	Storage layer only; deferred to Phase 2 here

Gap: prior work proves feasibility but rarely optimises on-chain retrieval. UniVerify V2.0 contributes an explicit string-keyed mapping for O(1) verification with hardcoded RBAC.

BASELINE Existing System

05 / 20

How academic verification works today.

- **Centralized databases (RDBMS).**
Records held in a single institutional server or department spreadsheet.
- **Manual verification.**
Employers email/call the university; staff look up and confirm — 7–14 days typical.
- **Paper / PDF certificates.**
Static documents that can be edited or forged without easy detection.
- **Custodial trust.**
Authenticity depends entirely on trusting the issuing authority and its staff.

AT A GLANCE

Latency

7–14 days

Trust

Centralized

Tamper-evidence

Weak

Availability

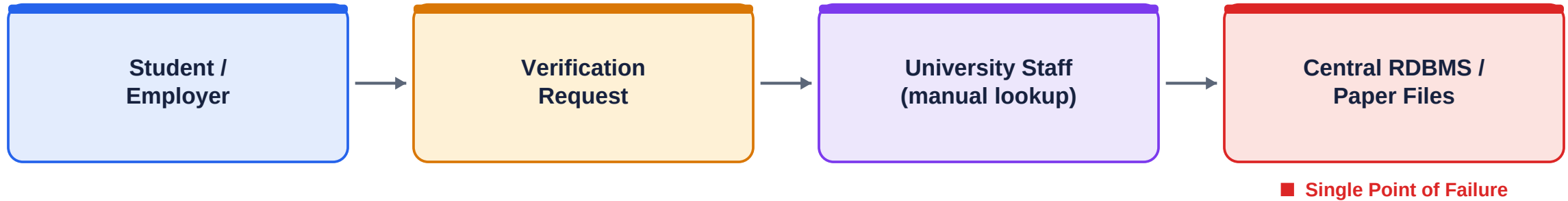
Single server

Audit

Manual / periodic

Architecture of Existing System

A centralized request-response pipeline with a single point of failure.



RISKS OF THE CENTRALIZED MODEL

- Server breach or insider edits can silently alter records.
- Downtime blocks all verification; no independent cross-check.
- Forged PDFs are hard to detect without manual institutional audit.

Drawbacks & Problem Statement

The specific deficiencies this project sets out to solve.

- **Single Point of Failure (SPOF).**

A central server is vulnerable to breach, insider modification and outage.

- **High latency & overhead.**

Manual lookups take 7–14 days and incur recurring administrative cost.

- **Forgery & tampering.**

Unanchored PDFs and paper can be visually manipulated; detection needs redundant audits.

- **No independent verification.**

Third parties cannot confirm a record without trusting (and contacting) the issuer.

PROBLEM STATEMENT

Design a decentralized, tamper-evident certificate verification system that offers instant, gasless, independently verifiable lookups while restricting issuance to an authorized authority.

Proposed System

A decentralized, constant-time verification protocol on Ethereum.

- **On-chain anchoring.**
Records written to a Solidity contract on Sepolia — immutable once mined.
- **O(1) keyspace mapping.**
mapping(string => Certificate) keyed by Registration Number; no search loops.
- **Role-Based Access Control.**
Constructor freezes the deployer as sole admin; unauthorized writes revert.
- **Gasless verification.**
Read-only view call via Ethers.js — instant and free.
- **Safety filter.**
Blank zero-state results shown as “Record Not Found”.

KEY BENEFITS

Immutable

O(1) lookup

No SPOF

Gasless reads

RBAC-secured

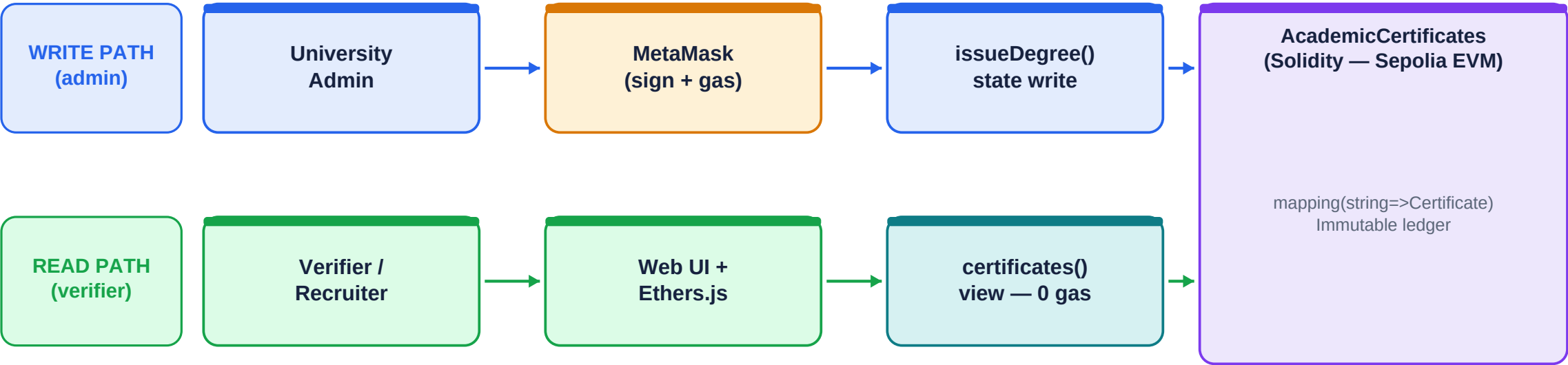
Global access

PHASE NOTE

Phase 1 uses manual entry in Remix with a mock hash placeholder; real SHA-256 / IPFS hashing and an autonomous AI agent arrive in Phase 2.

Architecture of Proposed System

Write path (admin, costs gas) and read path (verifier, gasless).



NETWORK

Public Ethereum Sepolia testnet — EVM execution, no central host. Writes are signed transactions (gas); reads are free view calls.

DESIGN

Module Diagram

10 / 20

The system decomposed into six cooperating modules.

UniVerify V2.0 — System Modules

Smart Contract

struct, mapping,
RBAC require()

Admin & Issuance

issueDegree(),
MetaMask signing

Web3 Provider

Ethers.js +
MetaMask bridge

Verification

view call +
safety filter

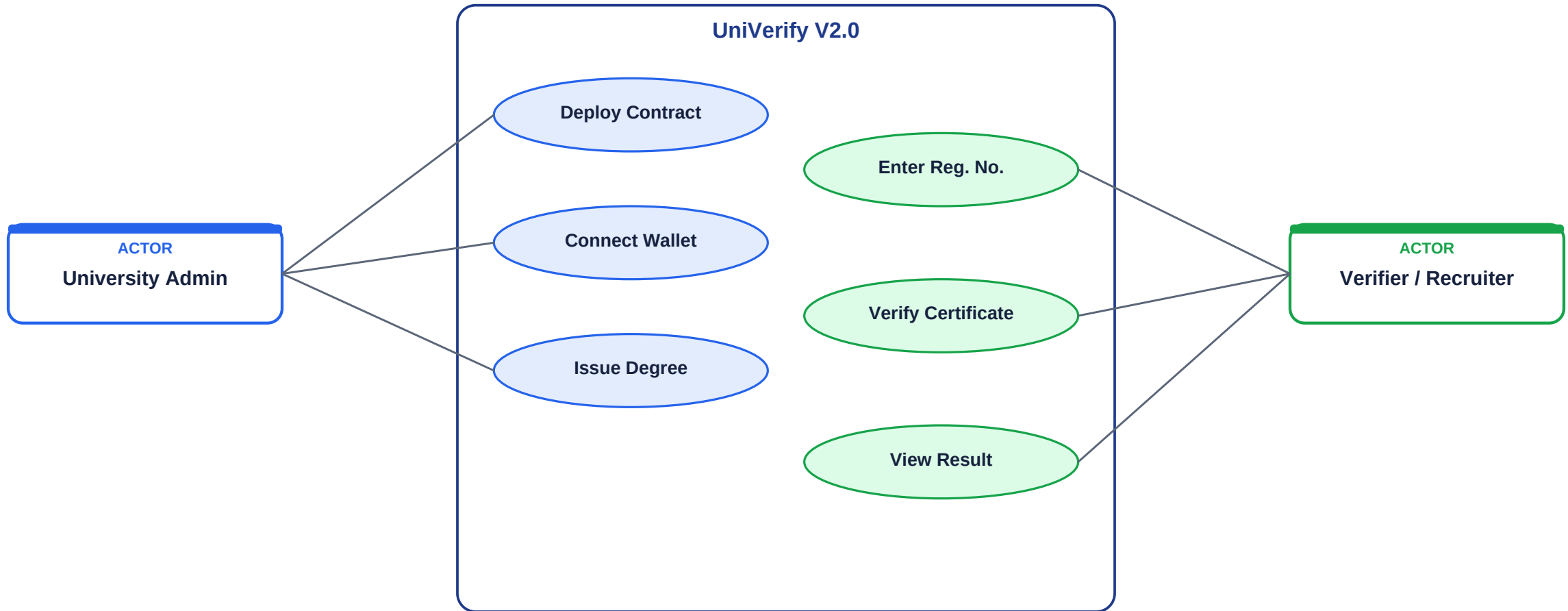
UI / Result

#regInput,
#result states

AI Agent (Phase 2)

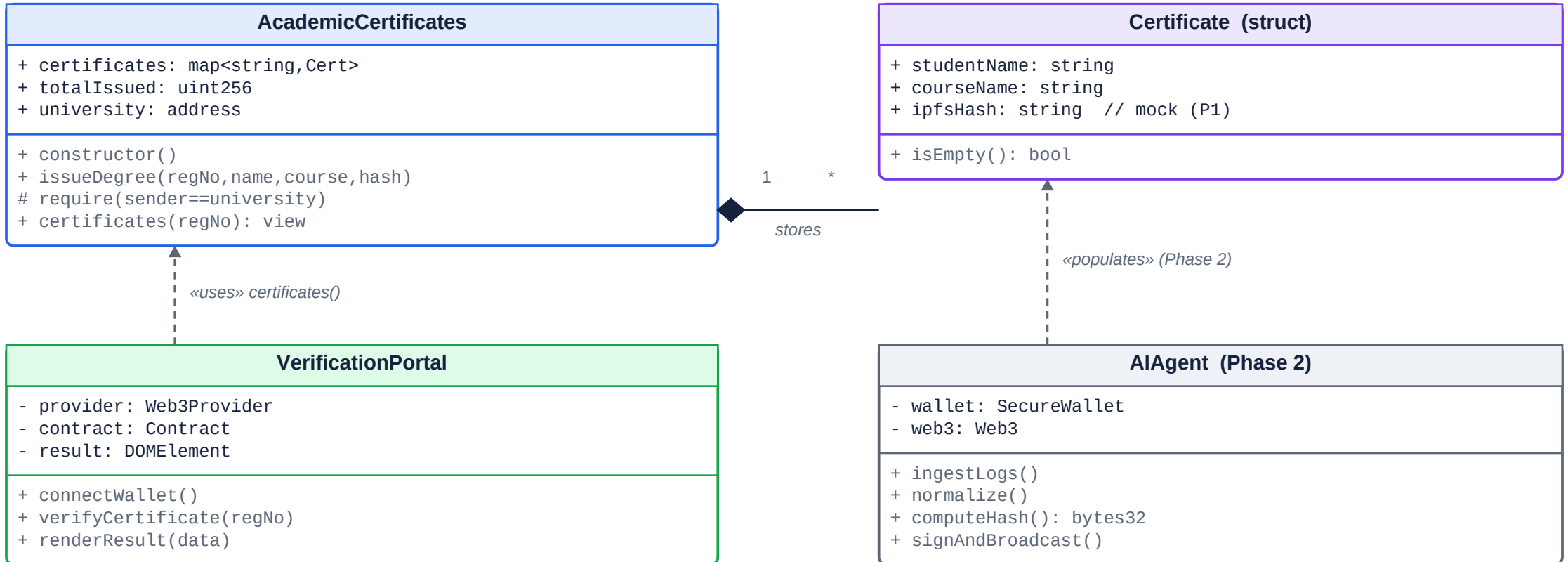
NLP + SHA-256 /
IPFS, auto-sign

Actors and the use cases they trigger inside the system.



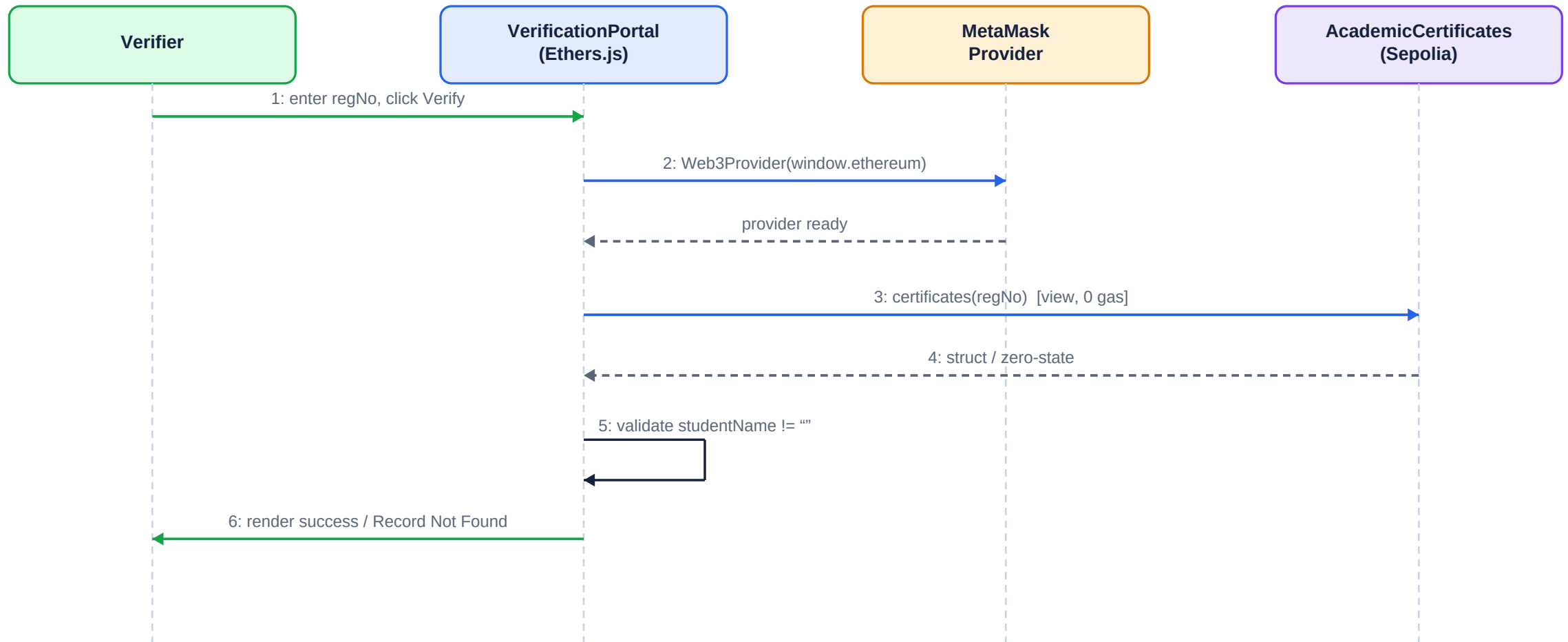
Class Diagram (Model Classes)

Domain model: contract, record struct, frontend, Phase-2 agent.



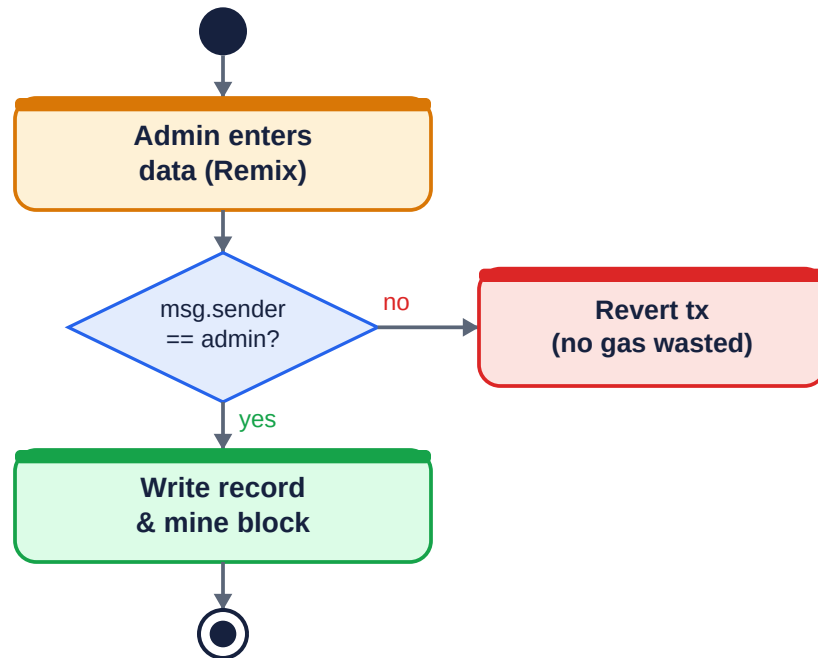
Sequence Diagram (Verification)

Gasless read-path message flow between client and contract.

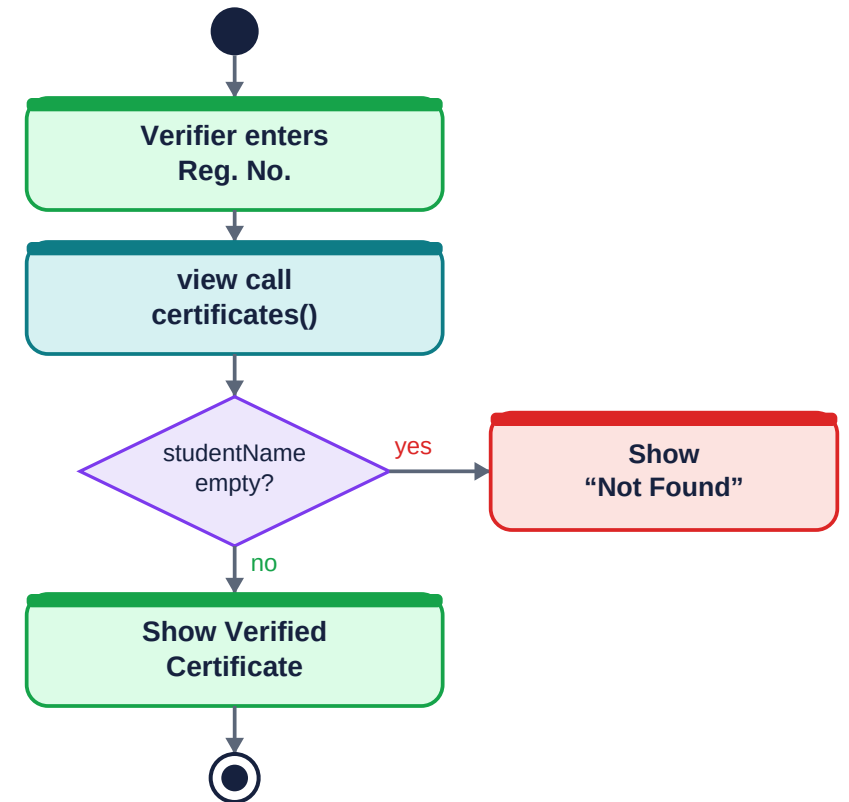


Issuance (write) and verification (read) control flow.

ISSUANCE (write path)

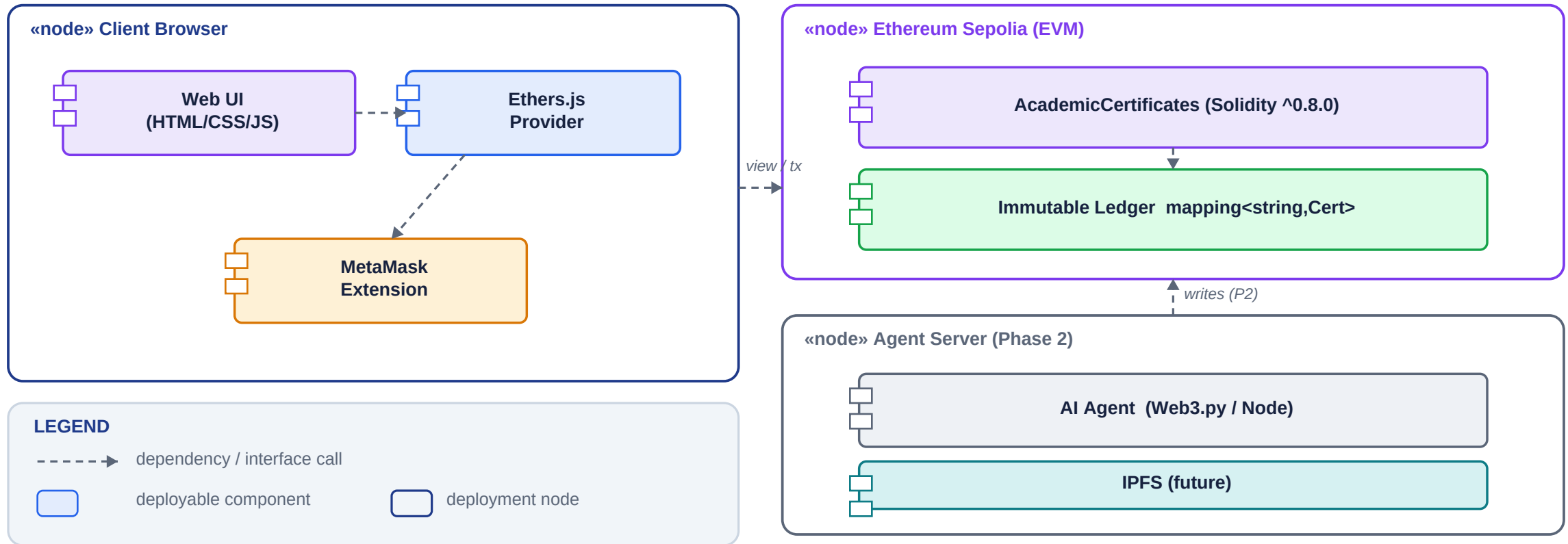


VERIFICATION (read path)



Component / Deployment Diagram

Components grouped by deployment node; dashed = dependency.



Algorithm Used in Proposed Solution

Constant-time write and read, with RBAC enforced before gas.

Algorithm 1 — issueDegree (WRITE)

```
Input : regNo, name, course, hash
Pre   : caller == university (RBAC)

1. require(msg.sender==university)
2.   else revert // EVM blocks tx
3. certificates[regNo] <-
    Certificate(name,course,hash)
4. totalIssued <- totalIssued + 1
Cost  : O(1) write (+ gas, Sepolia)
```

Algorithm 2 — verifyCertificate (READ)

```
Input : regNo

1. data <- contract.certificates(regNo)
    // gasless view call (Ethers.js)
2. if data.studentName == "" then
3.   render("Record Not Found")
4. else
5.   render(data) // verified card
Cost  : O(1) read (0 gas, instant)
```

WHY O(1)?

A hash-table state mapping resolves the storage slot for a Registration Number directly, with no iteration — unlike array/loop designs that are $O(N)$ and whose gas grows with the record count. RBAC `require()` rejects unauthorized writes before any gas is spent.

Output Screenshots

Representative UI mock-ups — replace with your real screenshots.

univerify.app/verify

UniVerify — Verify Certificate

2021CSE101

Verify

✓ Certificate Verified

Name: <Student Name>

Course: B.Tech CSE

Hash: a1b2c3... (mock, Phase 1)

univerify.app/verify

UniVerify — Verify Certificate

9999XYZ000

Verify

✗ Record Not Found

No certificate exists for this Registration Number. Zero-state result intercepted by the safety filter.

Remix IDE — issueDegree (Admin write, Phase 1)

_regNo: 2021CSE101

_name: Student Name

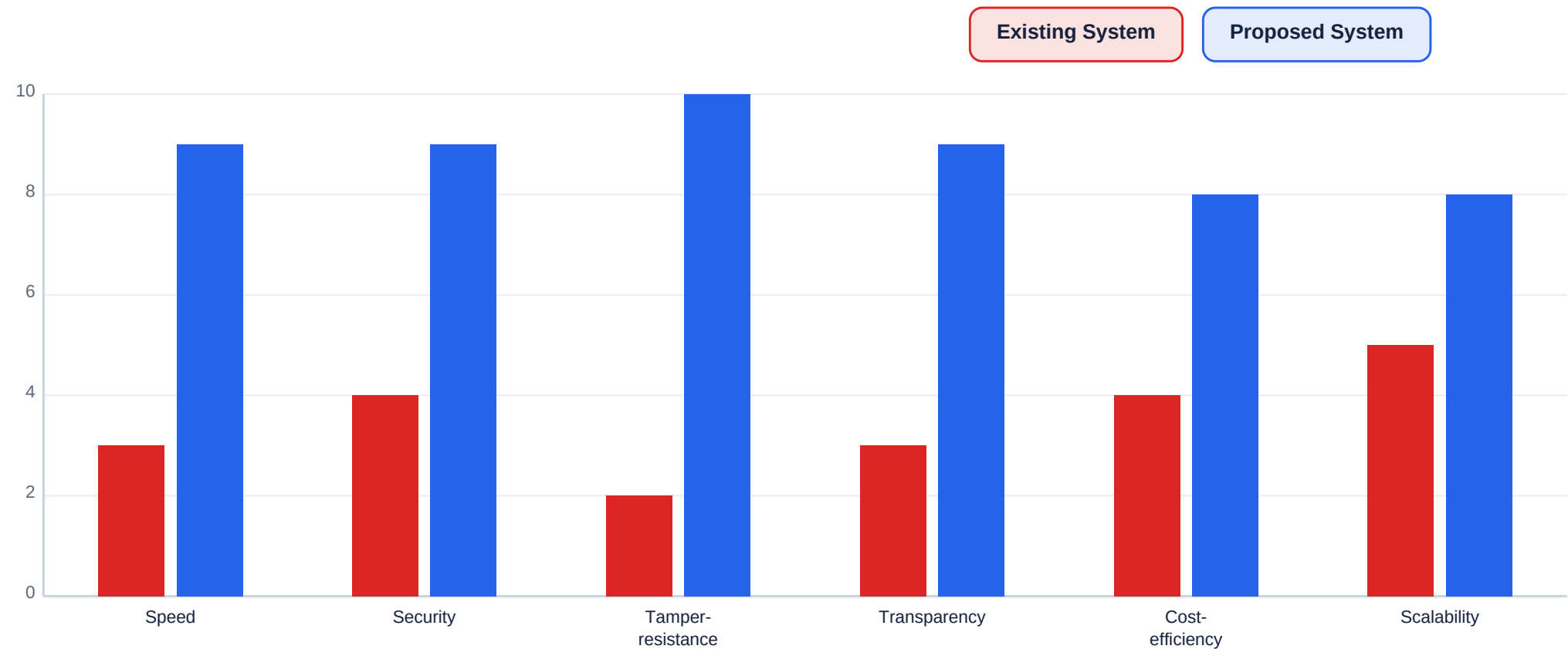
_course: B.Tech CSE

_hash: a1b2c3 (manual mock)

transact (sign)

✓ status: 0x1 mined — gas used (SepoliaETH)

Indicative qualitative scores (out of 10; higher is better).



Latency: ~7–14 days (existing) → ~seconds (proposed). Scores are indicative for comparison, not benchmarked units.

Future Enhancement & Conclusion

FUTURE ENHANCEMENTS

- **Phase 2 AI agent.**
NLP-normalized ingestion, SHA-256 / IPFS hashing, programmatic transaction signing.
- **Mainnet / Layer-2.**
Deploy on a low-fee L2 (e.g., Polygon) for production scale.
- **Revocation & expiry.**
On-chain credential revocation and validity windows.
- **Multi-institution registry.**
Federated admin roles for many universities.
- **QR + mobile DID wallet.**
Scan-to-verify and self-sovereign holder wallets.

CONCLUSION

UniVerify V2.0 turns certificate verification from a slow, centralized, forgery-prone process into an immutable, decentralized, constant-time lookup on a public blockchain.

The Phase-1 prototype validates state mutability, $O(1)$ string-keyed retrieval, RBAC guardrails and the gasless read path under a manual curation paradigm — with the data-fingerprint held as a mock placeholder.

Phase 2 closes the loop with autonomous, cryptographically-hashed issuance for an end-to-end trust-minimized system.

IEEE style — add the specific sources cited in your report.

- [1] S. Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System,” 2008.
- [2] V. Buterin, “Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform,” White Paper, 2014.
- [3] J. Benet, “IPFS — Content-Addressed, Versioned, P2P File System,” arXiv:1407.3561, 2014.
- [4] MIT Media Lab, “Blockcerts: An Open Standard for Blockchain Certificates,” 2016. Available: <https://www.blockcerts.org>
- [5] A. Grech, A. F. Camilleri, “Blockchain in Education,” Joint Research Centre, European Commission, 2017.
- [6] Ethereum Foundation, “Solidity Documentation (v0.8.x).” Available: <https://docs.soliditylang.org>
- [7] Ethers.js, “Documentation (v5.7).” Available: <https://docs.ethers.org/v5/>
- [8] ConsenSys, “MetaMask Developer Documentation.” Available: <https://docs.metamask.io>

Note: [1]–[5] are foundational works; replace or augment with the specific papers cited in your report.